Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 08

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad de Django REST Framework (serializers, vistas, ViewSets y routers) para exponer los datos de una aplicación empresarial como servicios web.**
2. **Implementar serializers y endpoints CRUD (vistas genéricas y ViewSets con routers) sobre modelos ya existentes, con validación y relaciones anidadas.**
3. **Aplicar búsqueda, ordenamiento y filtros por parámetro, y verificar los códigos de estado HTTP con un cliente de pruebas.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 8 (Django REST Framework: API y REST, métodos HTTP y códigos de estado, serializers, vistas genéricas, ViewSets, routers y filtros) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, Django REST Framework (pip install djangorestframework), un editor de código (Visual Studio Code), un cliente de pruebas de API (Thunder Client de VS Code, Postman o curl) y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada desde la Semana 3 (CRUD), ampliada con relaciones en la Semana 4, registrada en el Django Admin en la Semana 5, con Templates que heredan de base.html desde la Semana 6 y con el campo de estado y el QuerySet personalizado de la Semana 7, y, cuando corresponda, la investigación propia de siete entidades.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (serializers.py, api_views.py y las URLs) y de la respuesta de la API en el navegador o en el cliente de pruebas, mostrando el método HTTP y el código de estado obtenido.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Desde la Semana 3, cada estudiante ya cuenta con una aplicación con CRUD, relaciones, Django Admin, Templates con herencia y, desde la Semana 7, consultas avanzadas y un QuerySet personalizado. Hasta ahora esa aplicación entrega páginas HTML. En esta sesión no se agregan entidades nuevas: se exponen las que ya existen como una API REST que entrega datos en

formato JSON, para que otros programas (una app móvil, o React en la Semana 9) puedan consumirlas.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de las Semanas 4 a 7 y le agrega una API con Django REST Framework: serializers, vistas genéricas, ViewSets, routers, filtros y una acción propia. En la segunda parte, deberá hacer lo mismo sobre su investigación propia (las siete entidades registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que la API no reemplaza al ORM ni a las consultas que ya construyó: las reutiliza. El mismo Model, las mismas relaciones y el mismo QuerySet personalizado de la Semana 7 son los que alimentan a los serializers y a los ViewSets. En clase la teoría se demuestra con una aplicación de cafetería; cada estudiante la traslada a su propia aplicación con ayuda de la siguiente tabla de equivalencias.

Concepto trabajado en clase	Equivalente en su aplicación
Recurso simple con una regla de negocio (en clase: Producto, con precio mayor que cero)	Una entidad de su aplicación con campos simples y una regla que pueda validarse (precio, cantidad, capacidad, título).
Recurso con relaciones, de solo lectura (en clase: Pedido con sus detalles)	Su entidad principal junto con su relación 1:N, 1:1 o su modelo intermedio N:M.
Campo de estado para filtrar con ?estado= (en clase: estado de Pedido)	El campo de estado que trabajó en la Semana 7.
Método del QuerySet personalizado (en clase: pagados())	Un método del QuerySet personalizado de la Semana 7.

Cliente (navegador o Thunder Client)

→ URL de la API (ej. /api/productos/)

→ Router → ViewSet

→ Serializer (valida y convierte a JSON)

→ Manager / QuerySet personalizado

→ Django ORM

→ SQLite

→ Response (JSON y código de estado)

Al finalizar el laboratorio, el estudiante deberá ser capaz de crear serializers con validación, exponer un recurso con vistas genéricas y con ViewSets y routers, probar el CRUD verificando los códigos de estado HTTP, optimizar una API con relaciones anidadas y agregar búsqueda, ordenamiento, filtros por parámetro y una acción propia.

Enunciados

PARTE 1 — Una API REST sobre la aplicación de las Semanas 4 a 7

Ejercicio 1 — Instalar Django REST Framework e identificar los recursos

Abra la aplicación desarrollada hasta la Semana 7 e identifique dos recursos para exponer: un recurso simple (con la ayuda de la tabla de equivalencias) y otro con relaciones. Verifique que cuenta con el campo de estado y el QuerySet personalizado de la Semana 7; si no los tiene, créelos ahora. Instale el paquete con pip install djangorestframework, agregue 'rest_framework' a

INSTALLED_APPS y actualice requirements.txt. Documente con capturas los dos recursos elegidos y el archivo settings.py.

Respuesta:

Se eligieron dos recursos de la app vet. El recurso simple es el insumo, con la regla de negocio de que el stock inicial debe ser de al menos 1 unidad. El recurso con relaciones es la cita, que trae su veterinario (uno a muchos), su ficha clínica (uno a uno) y sus consumos de insumo (muchos a muchos con modelo intermedio).

Concepto de la clase	En la app vet
Recurso simple con regla de negocio	Insumo: stock inicial de al menos 1 unidad
Recurso con relaciones, de solo lectura	Cita con veterinario, ficha clínica y consumos
Campo de estado para ?estado=	Cita.estado (Pendiente, Confirmada, Atendida)
Método del QuerySet personalizado	CitaQuerySet.proximas() y con_estado()

El campo de estado y el QuerySet personalizado ya existían desde la Semana 7, así que no hubo que tocar los modelos ni crear migraciones. Se instaló el paquete con pip install djangorestframework (versión 3.18.3), se agregó rest_framework a INSTALLED_APPS y se añadió la línea a requirements.txt.

Instalación de Django REST Framework

```
PS ...\django_project> venv\Scripts\python.exe -m pip install djangorestframework
Successfully installed djangorestframework-3.18.3
PS ...\django_project> venv\Scripts\python.exe -m pip freeze
asgiref==3.12.1
Django==6.1.1
djangorestframework==3.18.3
sqlparse==0.6.0
tzdata==2026.4
```

config/settings.py y requirements.txt

```
# config/settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'vet',  # Reserva de citas de la veterinaria (Laboratorio 02)
    'biblioteca', # Catalogo de biblioteca comunitaria (Laboratorio 03 - Parte 2)
    'rest_framework',  # Django REST Framework (Laboratorio 08)
]

# requirements.txt
asgiref==3.12.1
Django==5.2.17
djangorestframework==3.18.3
sqlparse==0.6.0
tzdata==2026.3
```

Ejercicio 2 — Crear un ModelSerializer y serializar en el shell

Cree en serializers.py un ModelSerializer para el recurso simple. En el shell de Django (python manage.py shell), serialice un objeto y luego un QuerySet completo con many=True. Documente con capturas ambas salidas y explique con sus palabras por qué se necesita many=True en el segundo caso.

Respuesta:

En vet/serializers.py se creó InsumoSerializer, un ModelSerializer con los campos id, nombre, unidad y stock. En el shell se serializó primero la jeringa de 5 ml, que devolvió un diccionario, y luego todos los insumos con many=True, que devolvió una lista de diccionarios.

Se necesita many=True porque sin esa opción el serializer espera un solo objeto y trata de leer sus campos directamente. Un QuerySet es una colección y no tiene un nombre ni un stock propios; con many=True el serializer recorre la colección y convierte cada elemento.

vet/serializers.py — InsumoSerializer con validate_stock

```python
# vet/serializers.py
# Semana 8 - Django REST Framework.
# Un serializer convierte instancias del Model a JSON (serializar) y valida
# el JSON que llega antes de guardarlo (deserializar). Es el equivalente al
# Form de las Views HTML, pero para la API.

from rest_framework import serializers

from .models import Cita, ConsumoInsumo, FichaClinica, Insumo, Veterinario


# --- Ejercicio 2 y 3: recurso SIMPLE con una regla de negocio ----------
class InsumoSerializer(serializers.ModelSerializer):
    class Meta:
        model = Insumo
        fields = ['id', 'nombre', 'unidad', 'stock']

    # Ejercicio 3 - validate_<campo>: DRF lo llama solo dentro de is_valid().
    # Regla de negocio: un insumo se da de alta con al menos una unidad en
    # stock; registrar un insumo sin existencias no tiene sentido en almacen.
    def validate_stock(self, value):
        if value < 1:
            raise serializers.ValidationError(
                'El stock inicial debe ser de al menos 1 unidad.')
        return value
```

Shell de Django — serializar un objeto y un QuerySet (many=True)

```
PS ...\django_project\src> python manage.py shell
>>> from vet.models import Insumo
>>> from vet.serializers import InsumoSerializer
>>> insumo = Insumo.objects.get(nombre='Jeringa 5ml')
>>> InsumoSerializer(insumo).data
{'id': 1, 'nombre': 'Jeringa 5ml', 'unidad': 'unidad', 'stock': 100}
>>> InsumoSerializer(Insumo.objects.order_by('nombre'), many=True).data
[{'id': 4, 'nombre': 'Antibiótico inyectable', 'unidad': 'ampolla', 'stock': 30}, {'id': 5, 'nombre': 'Gasa estéril', 'unidad': 'paquete', 'stock': 45}, {'id': 1, 'nombre': 'Jeringa 5ml', 'unidad': 'unidad', 'stock': 100}, {'id': 6, 'nombre': 'Vacuna antirrábica', 'unidad': 'dosis', 'stock': 25}]
```

Ejercicio 3 — Deserializar y validar

Agregue al serializer una validación propia con un método validate_<campo> que exprese una regla de negocio (por ejemplo, precio mayor que cero o cantidad de al menos uno). En el shell, cree el serializer con datos (data=...) y pruebe un caso válido y uno inválido. Muestre el resultado de is_valid(), validated_data y errors, y guarde el caso válido con save().

Respuesta:

Se agregó el método validate_stock: un insumo nuevo debe registrarse con al menos 1 unidad en stock. DRF llama a este método dentro de is_valid().

Con stock 0, is_valid() devolvió False y errors mostró el mensaje de la regla. Con stock 40, is_valid() devolvió True, validated_data trajo los tres campos y save() creó el insumo Suero fisiológico con id 7.

Shell de Django — deserializar y validar

```
PS ...\django_project\src> python manage.py shell
>>> from vet.serializers import InsumoSerializer
>>> s = InsumoSerializer(data={'nombre': 'Suero fisiológico', 'unidad': 'frasco', 'stock': 0})
>>> s.is_valid()
False
>>> s.errors
{'stock': [ErrorDetail(string='El stock inicial debe ser de al menos 1 unidad.', code='invalid')]}
>>> s = InsumoSerializer(data={'nombre': 'Suero fisiológico', 'unidad': 'frasco', 'stock': 40})
>>> s.is_valid()
True
>>> s.validated_data
{'nombre': 'Suero fisiológico', 'unidad': 'frasco', 'stock': 40}
>>> s.save()
<Insumo: Suero fisiológico>
>>> s.data
{'id': 7, 'nombre': 'Suero fisiológico', 'unidad': 'frasco', 'stock': 40}
```

Ejercicio 4 — Exponer el recurso con vistas genéricas

Cree una vista con ListCreateAPIView y otra con RetrieveUpdateDestroyAPIView para el recurso simple, agregue sus URLs bajo /api/ y pruebe en el navegador la lista y el detalle (API navegable de DRF). Documente las URLs, el código y las capturas del navegador.

Respuesta:

En vet/api_views.py se crearon InsumoListCreate (ListCreateAPIView) e InsumoDetail (RetrieveUpdateDestroyAPIView), enlazadas en /api/insumos/ y /api/insumos/<pk>/. La API navegable mostró la lista con los métodos GET y POST, y el detalle con GET, PUT, PATCH y DELETE junto a un formulario para editar.

Las dos clases quedaron en api_views.py como referencia, pero sus rutas se comentaron en vet/api_urls.py porque en el Ejercicio 5 se reemplazaron por el ViewSet.

vet/api_views.py — vistas genéricas (Ejercicio 4)

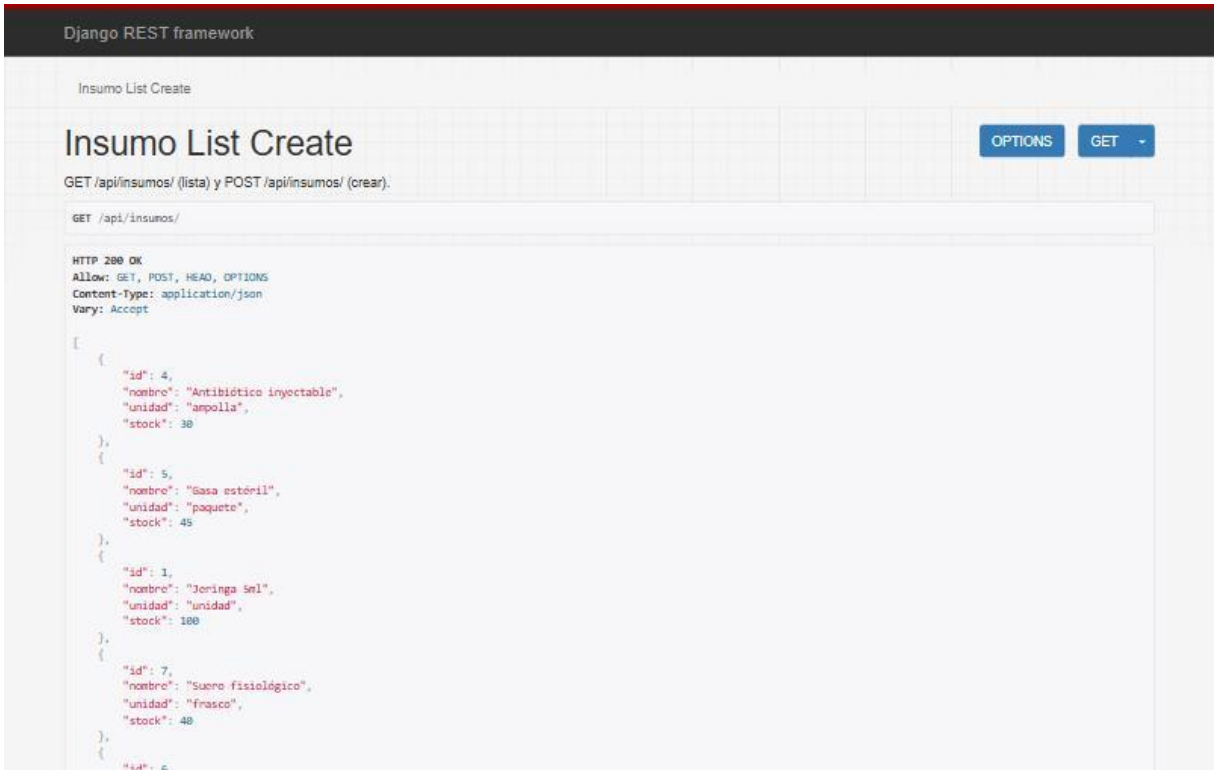
```python
# vet/api_views.py
# Semana 8 - las Views de la API. Igual que vet/views.py reciben el request,
# pero en lugar de renderizar un Template devuelven JSON con un codigo HTTP.
# Reutilizan el mismo Model, las mismas relaciones y el CitaQuerySet de la
# Semana 7.

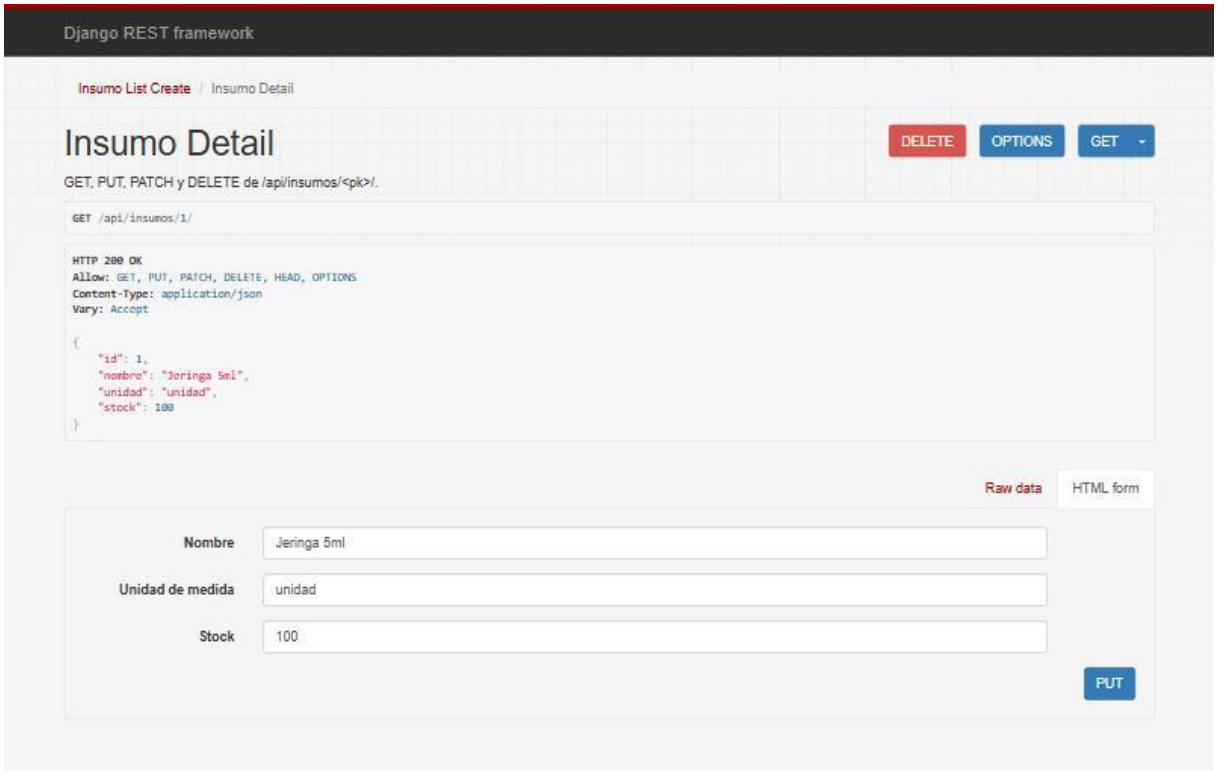
from rest_framework import filters, generics, viewsets
from rest_framework.decorators import action
from rest_framework.response import Response

from .models import Cita, Insumo
from .serializers import CitaSerializer, InsumoSerializer


# --- Ejercicio 4: vistas genericas (primera version) -------------------
# Quedan como referencia: en el Ejercicio 5 se reemplazaron por el
# InsumoViewSet de abajo y ya no estan enlazadas en api_urls.py.
class InsumoListCreate(generics.ListCreateAPIView):
    """GET /api/insumos/ (lista) y POST /api/insumos/ (crear)."""
    queryset = Insumo.objects.order_by('nombre')
    serializer_class = InsumoSerializer


class InsumoDetail(generics.RetrieveUpdateDestroyAPIView):
    """GET, PUT, PATCH y DELETE de /api/insumos/<pk>/."""
    queryset = Insumo.objects.all()
    serializer_class = InsumoSerializer
```





Ejercicio 5 — Reemplazar las vistas por un ViewSet y un router

Reemplace las dos vistas genéricas por un ModelViewSet registrado en un DefaultRouter. Abra /api/ y muestre los recursos registrados. Explique con sus palabras qué rutas y qué métodos HTTP se obtienen sin haberlos escrito a mano.

Respuesta:

Las vistas genéricas se reemplazaron por InsumoViewSet (ModelViewSet), registrado en un DefaultRouter en vet/api_urls.py; config/urls.py monta ese archivo en /api/. La raíz /api/ muestra los dos recursos registrados: insumos y citas.

Sin escribir ninguna ruta a mano, el router generó estas rutas y métodos:

Ruta	Métodos	Acción
/api/	GET	Raíz con los recursos
/api/insumos/	GET, POST	Listar y crear
/api/insumos/<pk>/	GET, PUT, PATCH, DELETE	Ver, reemplazar, modificar y borrar
/api/citas/	GET	Listar (solo lectura)
/api/citas/<pk>/	GET	Ver una cita
/api/citas/proximas/	GET	Acción propia del Ejercicio 8

El ViewSet reúne en una sola clase lo que antes eran dos vistas, y el router crea la URL de lista y la de detalle a partir del nombre con que se registra el recurso.

vet/api_views.py — InsumoViewSet

```python
# --- Ejercicio 5 y 8: recurso simple con ModelViewSet -------------------
class InsumoViewSet(viewsets.ModelViewSet):
    queryset = Insumo.objects.order_by('nombre')
    serializer_class = InsumoSerializer
    # Ejercicio 8 - ?search=texto y ?ordering=campo (o -campo).
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['nombre', 'unidad']
    ordering_fields = ['nombre', 'stock']
```

vet/api_urls.py y config/urls.py — DefaultRouter

```python
# vet/api_urls.py
# Semana 8 - rutas de la API. config/urls.py las monta bajo /api/.

from rest_framework.routers import DefaultRouter

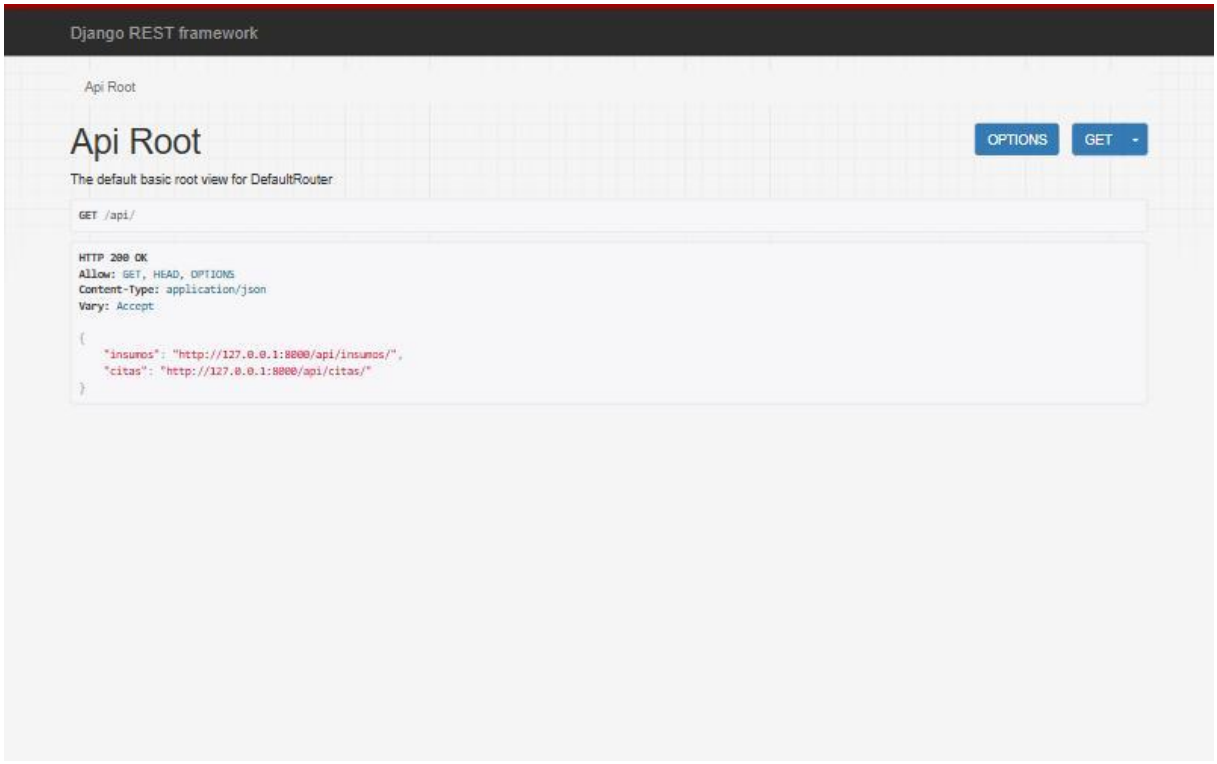
from . import api_views

# Ejercicio 5 - el router genera las rutas de lista y detalle de cada
# ViewSet, y ademas la raiz /api/ con los recursos registrados.
router = DefaultRouter()
router.register('insumos', api_views.InsumoViewSet, basename='insumo')
router.register('citas', api_views.CitaViewSet, basename='cita')

urlpatterns = router.urls

# Ejercicio 4 - version anterior con vistas genericas (reemplazada):
# from django.urls import path
# urlpatterns = [
#     path('insumos/', api_views.InsumoListCreate.as_view()),
#     path('insumos/<int:pk>/', api_views.InsumoDetail.as_view()),
# ]

# config/urls.py
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('vet.api_urls')),  # Laboratorio 08 - API REST
    path('', include('vet.urls')),
    path('biblioteca/', include('biblioteca.urls')),
]
```



Ejercicio 6 — Probar el CRUD y los códigos de estado

Con Thunder Client, Postman o curl, ejecute sobre el recurso simple: GET de la lista, GET de un detalle, POST válido, POST con datos inválidos, PUT o PATCH, DELETE y GET de un id inexistente. Complete una tabla con la petición, el código de estado obtenido y una captura de cada respuesta. Explique con sus palabras qué diferencia hay entre esta respuesta y la de un Template, y por qué esta API queda abierta hasta que se trabaje la seguridad.

Respuesta:

Las pruebas se hicieron con curl contra el servidor de desarrollo:

Petición	Código	Resultado
GET /api/insumos/	200 OK	Lista de los 5 insumos
GET /api/insumos/1/	200 OK	Jeringa 5ml, stock 100
POST /api/insumos/ (Collar isabelino, stock 12)	201 Created	Insumo creado con id 9
POST /api/insumos/ (nombre vacío, stock 0)	400 Bad Request	Errores en nombre y en stock
PATCH /api/insumos/9/ (stock 20)	200 OK	Stock actualizado a 20
DELETE /api/insumos/9/	204 No Content	Insumo borrado, sin cuerpo
GET /api/insumos/9/	404 Not Found	El insumo ya no existe

Un Template devuelve una página HTML ya armada para que la lea una persona. La API devuelve solo los datos en JSON, con un código de estado que dice si la operación salió bien, y quien la consume (una app móvil o React) decide cómo mostrarlos. Por eso un error de validación llega como un JSON con código 400 y no como un formulario con mensajes.

La API queda abierta porque no se configuró autenticación ni permisos: DRF usa por defecto AllowAny, así que cualquiera que conozca la URL puede crear, modificar o borrar insumos sin iniciar sesión. Queda así hasta que se trabaje la seguridad.

Terminal — curl

```
PS> curl -X GET "http://127.0.0.1:8000/api/insumos/"
HTTP 200
[
  {
    "id": 4,
    "nombre": "Antibiótico inyectable",
    "unidad": "ampolla",
    "stock": 30
  },
  {
    "id": 5,
    "nombre": "Gasa estéril",
    "unidad": "paquete",
    "stock": 45
  },
  {
    "id": 1,
    "nombre": "Jeringa 5ml",
    "unidad": "unidad",
    "stock": 100
  },
  {
    "id": 7,
    "nombre": "Suero fisiológico",
    "unidad": "frasco",
    "stock": 40
  },
  {
    "id": 6,
    "nombre": "Vacuna antirrábica",
    "unidad": "dosis",
    "stock": 25
  }
]
```

Terminal — curl

```
PS> curl -X GET "http://127.0.0.1:8000/api/insumos/1/"
HTTP 200
{
  "id": 1,
  "nombre": "Jeringa 5ml",
  "unidad": "unidad",
  "stock": 100
}
```

Terminal — curl

```
PS> curl -X POST "http://127.0.0.1:8000/api/insumos/" -H "Content-Type: application/json" -d '{"nombre": "Collar isabelino", "unidad": "unidad", "stock": 12}'
HTTP 201
{
  "id": 9,
  "nombre": "Collar isabelino",
  "unidad": "unidad",
  "stock": 12
}
```

```
Terminal — curl

PS> curl -X POST "http://127.0.0.1:8000/api/insumos/" -H "Content-Type: application/json" -d '{"nombre": "", "unidad": "unidad", "stock": 0}'
HTTP 400
{
  "nombre": [
    "Este campo no puede estar en blanco."
  ],
  "stock": [
    "El stock inicial debe ser de al menos 1 unidad."
  ]
}
```

```
Terminal — curl

PS> curl -X PATCH "http://127.0.0.1:8000/api/insumos/9/" -H "Content-Type: application/json" -d '{"stock": 20}'
HTTP 200
{
  "id": 9,
  "nombre": "Collar isabelino",
  "unidad": "unidad",
  "stock": 20
}
```

```
Terminal — curl

PS> curl -X DELETE "http://127.0.0.1:8000/api/insumos/9/"
HTTP 204
(sin cuerpo)
```

```
Terminal — curl

PS> curl -X GET "http://127.0.0.1:8000/api/insumos/9/"
HTTP 404
{
  "detail": "No Insumo matches the given query."
}
```

Ejercicio 7 — Un recurso de solo lectura con relaciones anidadas

Cree un serializer para el recurso con relaciones que incluya datos de la entidad relacionada de forma anidada (por ejemplo, un pedido con sus detalles) y exponga el recurso con un ReadOnlyModelViewSet. Escriba get_queryset() con select_related() o prefetch_related() según corresponda. Mida cuántas consultas ejecuta la serialización de todos los registros con Pedido.objects.all() (o su equivalente) y con el queryset optimizado, usando connection.queries, y documente el antes y el después.

Respuesta:

CitaSerializer anida tres serializers: el del veterinario (clave foránea), el de la ficha clínica (uno a uno, sale null si la cita no tiene ficha) y el de los consumos con many=True (modelo intermedio, con el nombre del insumo, la cantidad, el precio y el subtotal). El recurso se expone con CitaViewSet, un ReadOnlyModelViewSet, por lo que solo admite GET.

En get_queryset() se usa select_related para el veterinario y la ficha, y prefetch_related para los consumos y su insumo. Se midió con connection.queries la serialización de las 5 citas:

Queryset	Consultas
Cita.objects.all()	24
Con select_related + prefetch_related	3

Sin optimizar se hace 1 consulta para las citas, 5 para los veterinarios, 5 para las fichas, 5 para los consumos y 8 para el insumo de cada consumo (24 en total). Optimizado son 3: una con JOIN para citas, veterinarios y fichas, una para los consumos y una para los insumos.

vet/serializers.py — serializers anidados

```
class ConsumoInsumoSerializer(serializers.ModelSerializer):
    # Datos del Insumo (otro extremo del N:M) y la @property subtotal.
    insumo = serializers.CharField(source='insumo.nombre')
    subtotal = serializers.DecimalField(max_digits=9, decimal_places=2,
                                        read_only=True)

    class Meta:
        model = ConsumoInsumo
        fields = ['insumo', 'cantidad', 'precio_unitario', 'subtotal']


class CitaSerializer(serializers.ModelSerializer):
    # 1:N  -> el veterinario completo en lugar de solo su id.
    veterinario = VeterinarioSerializer(read_only=True)
    # 1:1  -> la ficha clinica; null si la cita aun no fue atendida.
    ficha = FichaClinicaSerializer(read_only=True, allow_null=True)
    # N:M  -> los consumos del modelo intermedio (cantidad y precio).
    consumos = ConsumoInsumoSerializer(many=True, read_only=True)

    class Meta:
        model = Cita
        fields = ['id', 'mascota', 'dueno', 'servicio', 'fecha', 'hora',
                  'estado', 'veterinario', 'ficha', 'consumos']
```

vet/api_views.py — CitaViewSet (get_queryset, ?estado= y @action)

```
# --- Ejercicio 7 y 8: recurso con relaciones, solo lectura -------------
class CitaViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = CitaSerializer

    def get_queryset(self):
        # select_related: veterinario (FK) y ficha (1:1) con JOIN.
        # prefetch_related: consumos y su insumo, en dos consultas extra.
        citas = (Cita.objects
                 .select_related('veterinario', 'ficha')
                 .prefetch_related('consumos__insumo')
                 .order_by('fecha', 'hora'))
        # Ejercicio 8 - filtro por parametro: /api/citas/?estado=Pendiente
        estado = self.request.query_params.get('estado')
        if estado:
            citas = citas.con_estado(estado)
        return citas

    # Ejercicio 8 - accion propia: GET /api/citas/proximas/
    # Reutiliza CitaQuerySet.proximas() de la Semana 7 sobre el queryset ya
    # optimizado (y filtrado por ?estado= si se envio).
    @action(detail=False)
    def proximas(self, request):
        citas = self.get_queryset().proximas()
        serializer = self.get_serializer(citas, many=True)
        return Response(serializer.data)
```

medir_api_n1.py

```python
# medir_api_n1.py
# Laboratorio 08 - Ejercicio 7: consultas que ejecuta la serializacion
# anidada de TODAS las citas, antes y despues de optimizar el queryset.
# Uso:  python manage.py shell -c "exec(open('medir_api_n1.py', encoding='utf-8').read())"
from django.db import connection, reset_queries
from django.test.utils import CaptureQueriesContext

from vet.models import Cita
from vet.serializers import CitaSerializer

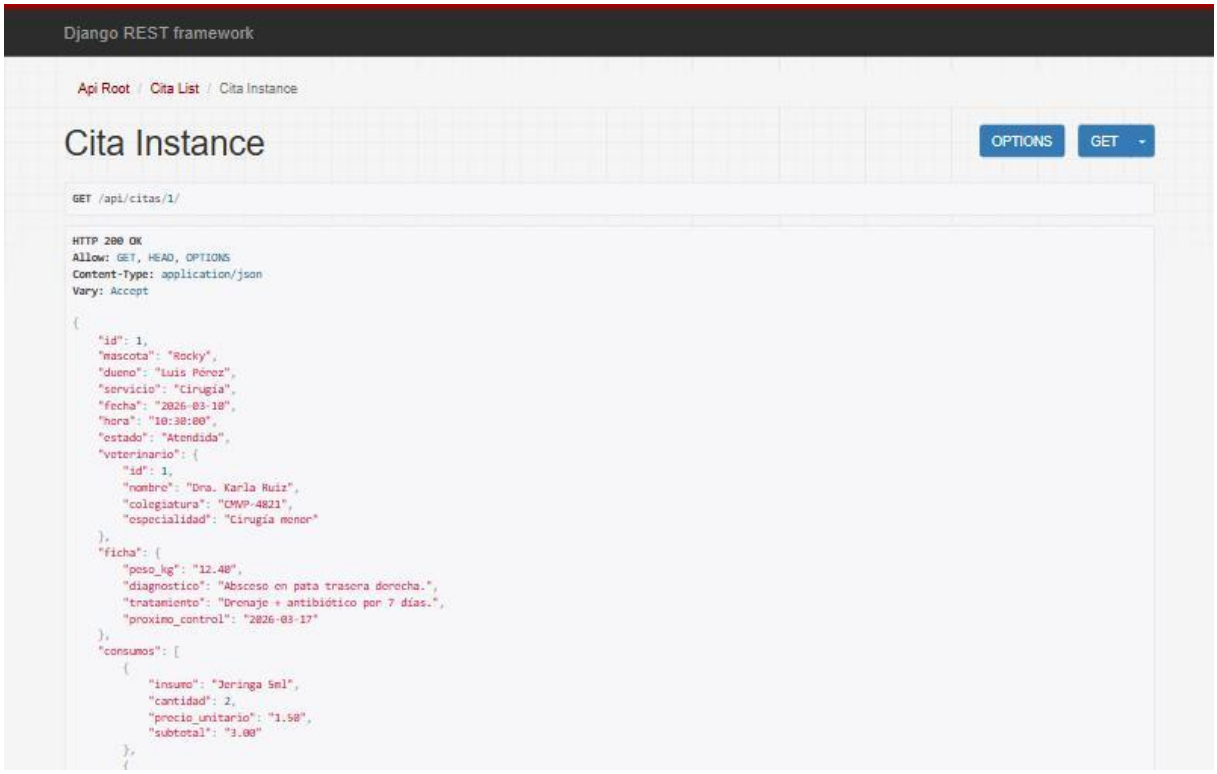
# connection.queries solo se llena con DEBUG=True; CaptureQueriesContext
# fuerza el registro para que la medicion no dependa de settings.
with CaptureQueriesContext(connection):
    reset_queries()
    CitaSerializer(Cita.objects.all(), many=True).data
    antes = len(connection.queries)

    reset_queries()
    optimizado = (Cita.objects
                  .select_related('veterinario', 'ficha')
                  .prefetch_related('consumos__insumo'))
    CitaSerializer(optimizado, many=True).data
    despues = len(connection.queries)

print('Citas serializadas:', Cita.objects.count())
print('Consultas SIN optimizar (Cita.objects.all()):', antes)
print('Consultas CON select_related + prefetch_related:', despues)
```

Terminal — consultas antes y después

```
PS ...\django_project\src> python manage.py shell -c "exec(open('medir_api_n1.py', encoding='utf-8').read())"
Citas serializadas: 5
Consultas SIN optimizar (Cita.objects.all()): 24
Consultas CON select_related + prefetch_related: 3
```



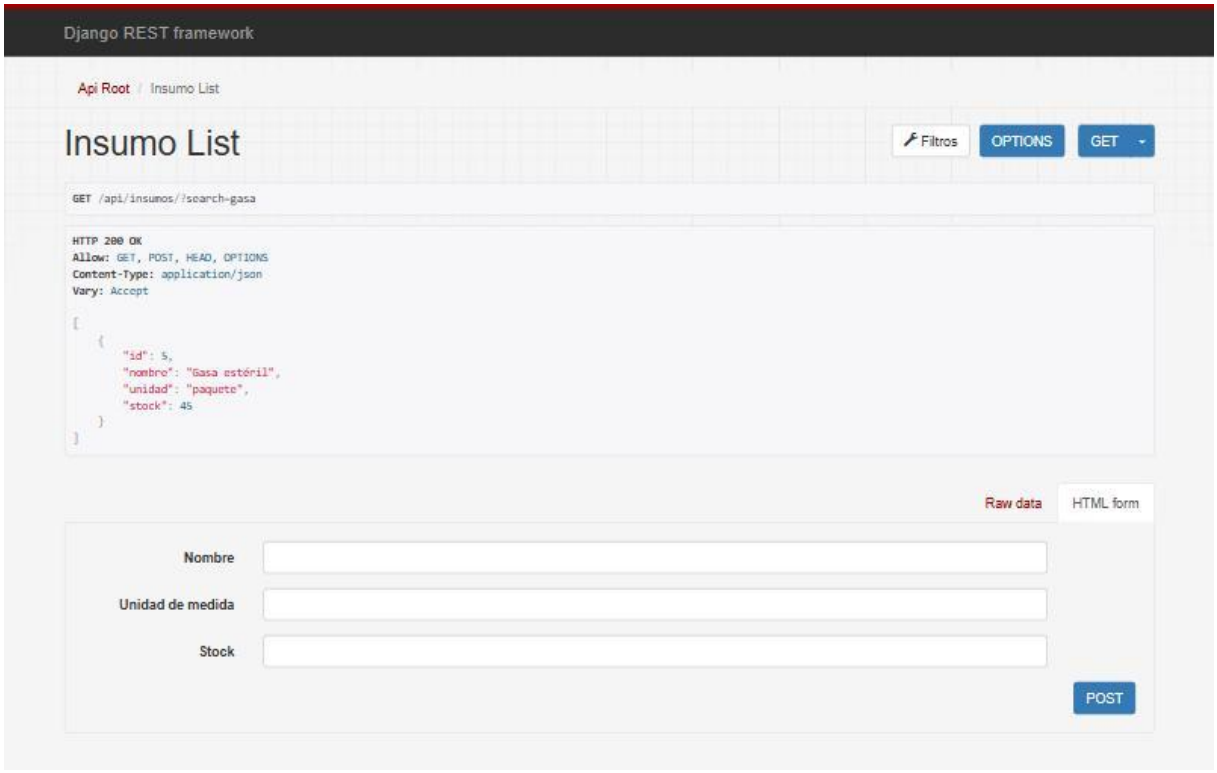
Ejercicio 8 — Búsqueda, ordenamiento, filtro por parámetro y una acción propia

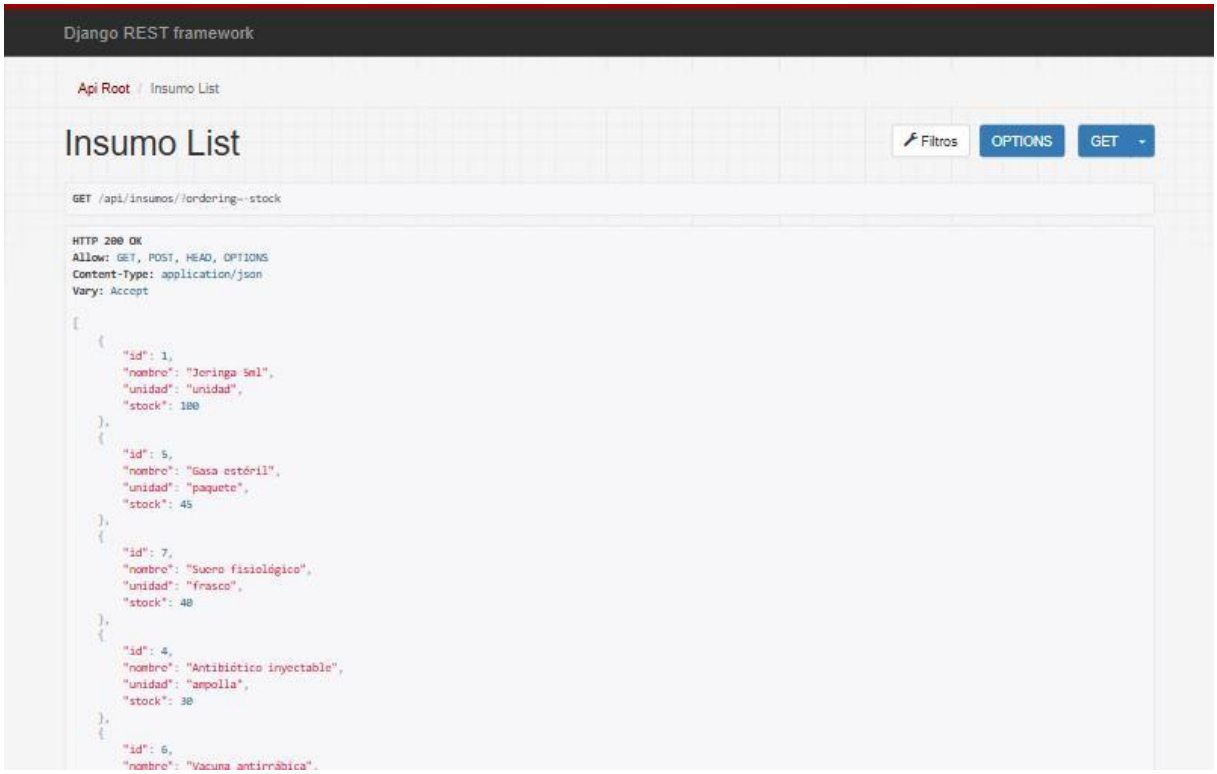
En el ViewSet del recurso simple agregue SearchFilter y OrderingFilter (search_fields y ordering_fields). En el ViewSet del recurso con relaciones, agregue en get_queryset() un filtro por el parámetro ?estado= (o el campo de estado de su aplicación) y una acción con @action(detail=False) que reutilice un método del QuerySet personalizado de la Semana 7. Documente al menos tres URLs con sus resultados, incluyendo una que combine parámetros con &.

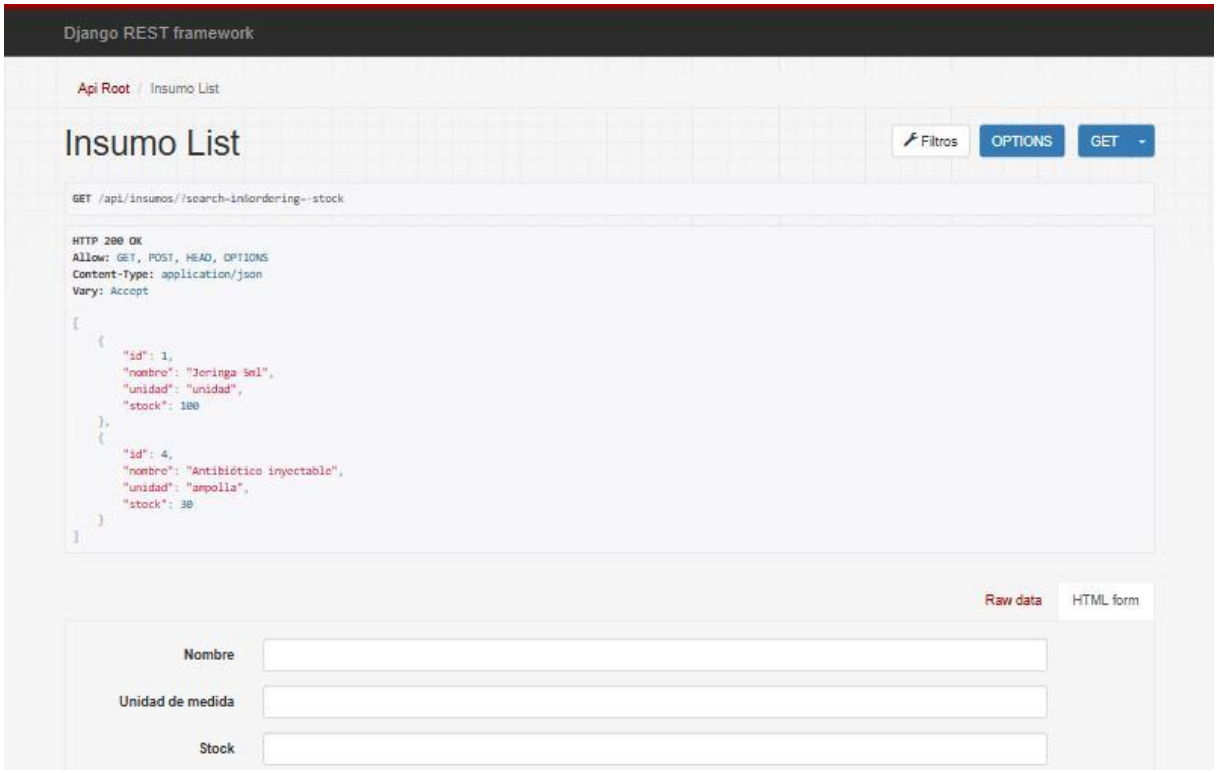
Respuesta:

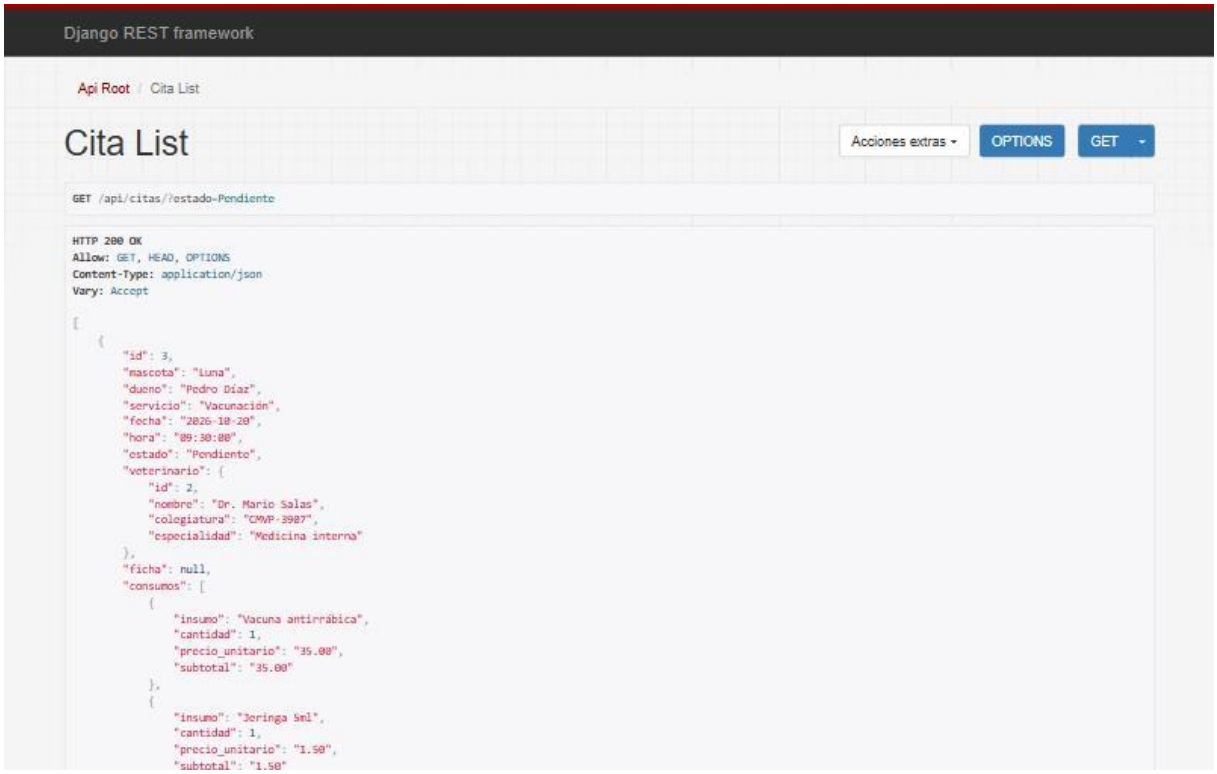
InsumoViewSet tiene SearchFilter (busca en nombre y unidad) y OrderingFilter (ordena por nombre o stock). En CitaViewSet, get_queryset() lee ?estado= y lo aplica con con_estado(), y la acción proximas, con @action(detail=False), reutiliza CitaQuerySet.proximas() de la Semana 7 sobre el mismo queryset, así que también respeta ?estado=.

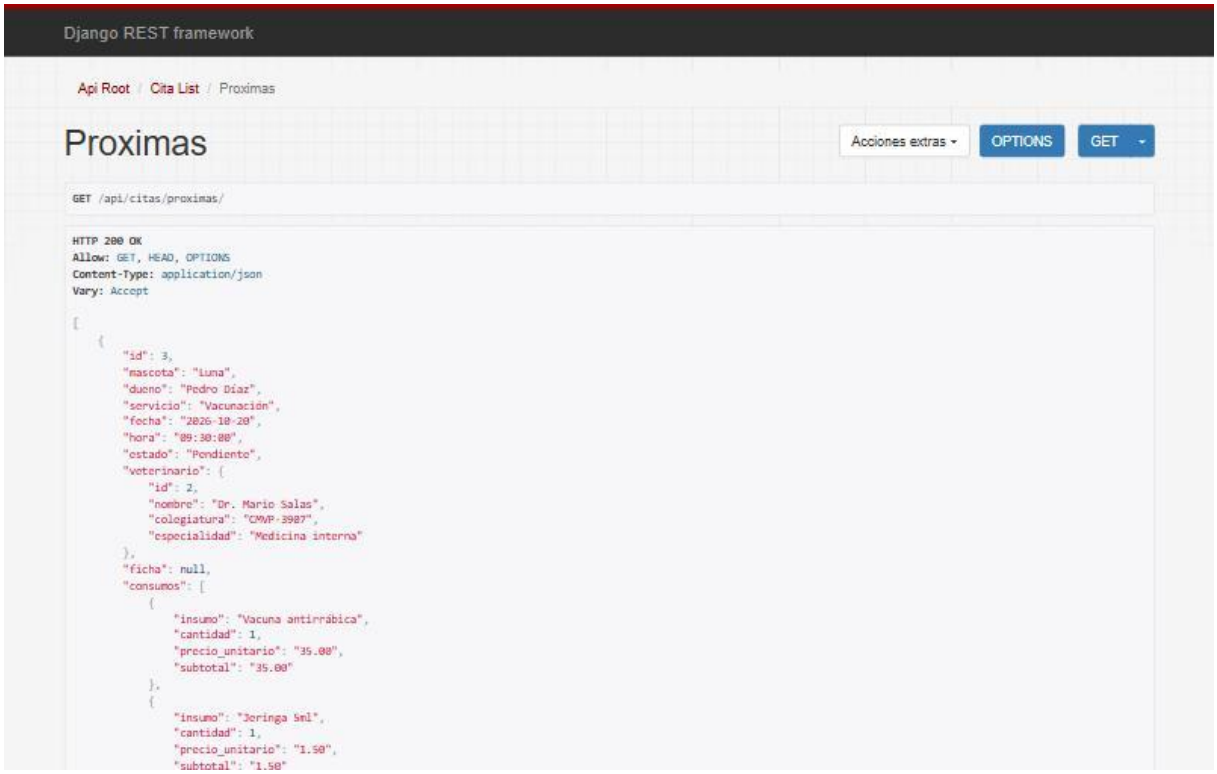
URL	Resultado
/api/insumos/?search=gasa	Gasa estéril
/api/insumos/?ordering=-stock	Jeringa (100), Gasa (45), Suero (40), Antibiótico (30), Vacuna (25)
/api/insumos/?search=in&ordering=-stock	Jeringa 5ml y Antibiótico inyectable
/api/citas/?estado=Pendiente	Luna
/api/citas/proximas/	Luna (20/10) y Toby (21/10)
/api/citas/proximas/?estado=Confirmada	Toby

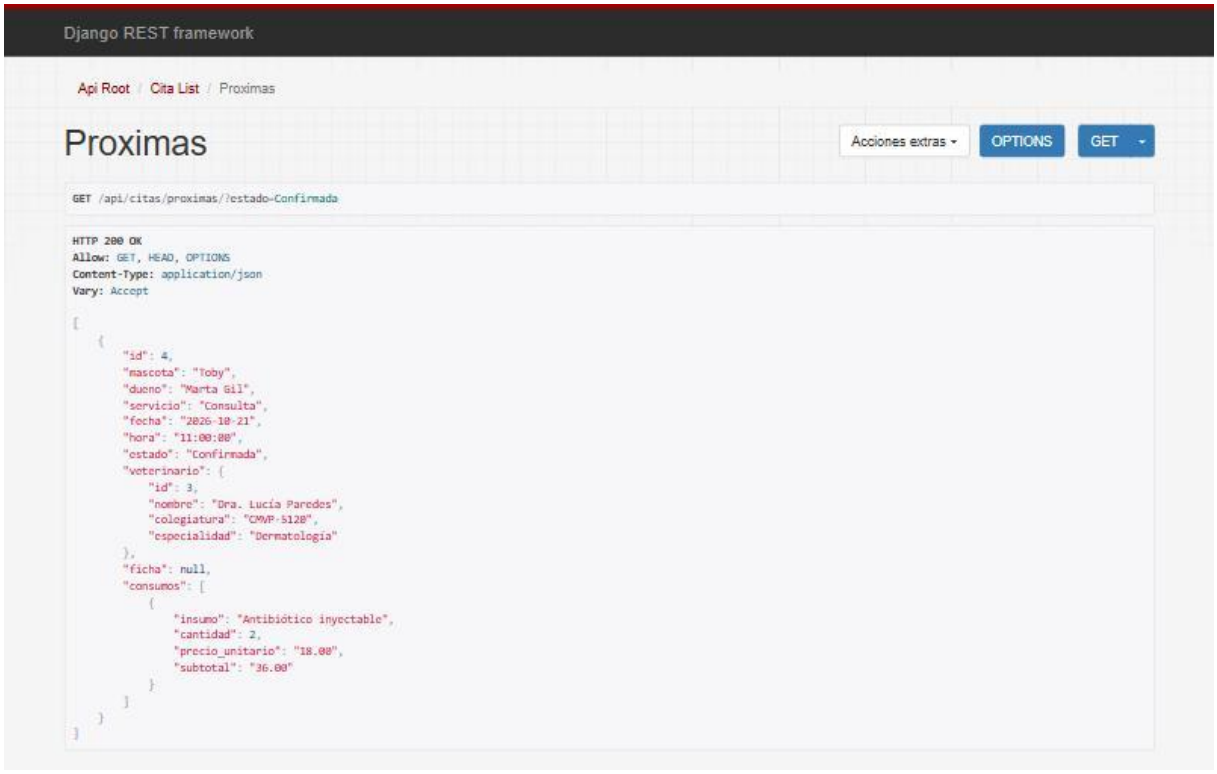

Django REST framework
Api Root / Insumo List
Insumo List
Filtros
OPTIONS
GET
GET /api/insumos/?search=gasa
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
[
{
"id": 5,
"nombre": "Gasa estéril",
"unidad": "paquete",
"stock": 45
}
]
Raw data
HTML form
Nombre
Unidad de medida
Stock
POST


Django REST framework
Api Root / Insumo List
Insumo List
Filtros
OPTIONS
GET
GET /api/insumos/?ordering=-stock
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
[
{
"id": 1,
"nombre": "Jeringa 5ml",
"unidad": "unidad",
"stock": 100
},
{
"id": 5,
"nombre": "Gasa estéril",
"unidad": "paquete",
"stock": 45
},
{
"id": 7,
"nombre": "Suero fisiológico",
"unidad": "frasco",
"stock": 40
},
{
"id": 4,
"nombre": "Antibiótico inyectable",
"unidad": "ampolla",
"stock": 30
},
{
"id": 6,
"nombre": "Vacuna antirrábica",


Django REST framework
Api Root / Insumo List
Insumo List
Filtros
OPTIONS
GET
GET /api/insumos/?search-inlordering=-stock
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
[
{
"id": 1,
"nombre": "Jeringa 5ml",
"unidad": "unidad",
"stock": 100
},
{
"id": 4,
"nombre": "Antibiótico inyectable",
"unidad": "ampolla",
"stock": 30
}
]
Raw data
HTML form
Nombre
Unidad de medida
Stock


Django REST framework
Api Root / Cita List
Cita List
Acciones extras
OPTIONS
GET
GET /api/citas/?estado=Pendiente
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
[
{
"id": 3,
"mascota": "Luna",
"dueno": "Pedro Diaz",
"servicio": "Vacunación",
"fecha": "2026-10-20",
"hora": "09:30:00",
"estado": "Pendiente",
"veterinario": {
"id": 2,
"nombre": "Dr. Mario Salas",
"colegiatura": "CMVP-3987",
"especialidad": "Medicina interna"
},
"ficha": null,
"consumos": [
{
"insumo": "Vacuna antirrábica",
"cantidad": 1,
"precio_unitario": "35.00",
"subtotal": "35.00"
},
{
"insumo": "Jeringa 5ml",
"cantidad": 1,
"precio_unitario": "1.50",
"subtotal": "1.50"





La API resuelve lo que no resuelven los Templates ni el Admin: los Templates muestran páginas a una persona y el Admin es una herramienta interna, mientras que la API entrega los mismos datos en JSON para que otro programa los lea o escriba. Le falta todavía autenticación y permisos, paginar las listas y configurar CORS para que React pueda llamarla desde otro puerto.

PARTE 2 — Una API REST sobre la investigación propia (registrada en el Admin en la Semana 5)

Esta parte se desarrolla sobre la investigación propia y está a cargo de Lucas Inga.

A partir de aquí trabajará sobre la base de datos de su investigación propia: las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es aplicar las mismas técnicas sobre ese modelo de datos, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Identificar los recursos de la investigación propia

Complete la tabla de equivalencias con su investigación: indique qué entidad expondrá como recurso simple con una regla de validación, qué entidad expondrá como recurso de solo lectura con relaciones, qué campo de estado usará para filtrar y qué método del QuerySet personalizado reutilizará en la acción. Si algún campo o método no existe, agréguelo antes de continuar.

Respuesta:

Se trabajó con la app biblioteca, expuesta en /api/biblioteca/ (el prefijo /api/ ya lo usa la API de vet). El campo de estado y el QuerySet personalizado ya existían de la Semana 7.

Elemento pedido	**En la investigación (biblioteca)**
Recurso simple con regla de validación	Libro: el año de publicación debe estar entre 1450 y el año actual
Recurso de solo lectura con relaciones	Prestamo, con socio y libro anidados
Campo de estado para filtrar	Prestamo.estado (?estado=)
Método del QuerySet reutilizado en la acción	PrestamoQuerySet.con_multa()

Checklist antes de continuar — Parte 2

Criterio	**Verificación**
1. Django REST Framework está instalado, 'rest_framework' está en INSTALLED_APPS y requirements.txt está actualizado.	Captura de settings.py y de requirements.txt.
2. Al menos tres recursos de tu investigación tienen un ModelSerializer.	Código de serializers.py.
3. Al menos un serializer tiene una validación propia (validate_<campo>), probada con un dato válido y uno inválido.	Capturas de is_valid() y errors.
4. Al menos dos recursos se exponen con ModelViewSet y DefaultRouter bajo /api/.	Código de api_views.py y captura de /api/.
5. Se verificaron con un cliente de pruebas los códigos 200, 201, 400, 404 y 204.	Tabla de peticiones y códigos con capturas.
6. Existe un recurso de solo lectura con relación anidada y get_queryset() optimizado; se midió el número de consultas antes y después.	Capturas de len(connection.queries).
7. Al menos un ViewSet tiene SearchFilter y OrderingFilter funcionando.	Capturas de las URLs con ?search= y ?ordering=.
8. Hay un filtro por parámetro (?campo=) y una @action que reutiliza un método del QuerySet personalizado de la Semana 7.	Código + captura de la respuesta.
9. Se explica por escrito qué resuelve la API frente a los Templates y al Admin, y qué le falta todavía.	Párrafo de justificación.

Ejercicio 10 — Serializers y validación en la investigación propia

Cree ModelSerializers para al menos tres recursos de su investigación. En al menos uno agregue una validación propia con validate_<campo> y pruébela en el shell con un dato válido y uno inválido, mostrando is_valid() y errors.

Respuesta:

serializers.py tiene ModelSerializer para tres recursos: Categoria, Libro y Prestamo (más dos resúmenes anidados, de Socio y de Libro). LibroSerializer incluye validate_anio_publicacion. Prueba en el shell: un libro de 2020 es válido (is_valid() True, errors vacío) y uno de 1200 es inválido (is_valid() False y el mensaje de la regla en errors).

biblioteca/serializers.py (líneas 1-46)

```python
from datetime import date

from rest_framework import serializers

from .models import Categoria, Libro, Prestamo, Socio


class CategoriaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Categoria
        fields = ['id', 'nombre', 'descripcion']


class LibroSerializer(serializers.ModelSerializer):
    class Meta:
        model = Libro
        fields = ['id', 'titulo', 'autor', 'isbn', 'anio_publicacion',
                  'copias', 'categoria']

    def validate_anio_publicacion(self, value):
        if value < 1450 or value > date.today().year:
            raise serializers.ValidationError(
                'El año de publicación debe estar entre 1450 y el año actual.')
        return value


class SocioResumenSerializer(serializers.ModelSerializer):
    class Meta:
        model = Socio
        fields = ['id', 'nombre', 'dni']


class LibroResumenSerializer(serializers.ModelSerializer):
    class Meta:
        model = Libro
        fields = ['id', 'titulo', 'autor']


class PrestamoSerializer(serializers.ModelSerializer):
    socio = SocioResumenSerializer(read_only=True)
    libro = LibroResumenSerializer(read_only=True)

    class Meta:
        model = Prestamo
        fields = ['id', 'socio', 'libro', 'fecha_prestamo',
                  'fecha_devolucion_prevista', 'fecha_devolucion_real',
```

```
Shell: caso valido e invalido
----------------------------
--- Caso valido ---
is_valid: True
validated_data: {'titulo': 'Libro de prueba API', 'autor': 'Autor Prueba', 'isbn': '978-API-0001', 'anio_publicacion': 2020, 'copias': 2, 'categoria':
errors: {}
guardado: Libro de prueba API (Novela)
--- Caso invalido (año 1200) ---
is_valid: False
errors: {'anio_publicacion': [ErrorDetail(string='El año de publicación debe estar entre 1450 y el año actual.', code='invalid')]}
```

Ejercicio 11 — ViewSets, router y códigos de estado en la investigación propia

Exponga al menos dos recursos con ModelViewSet y un DefaultRouter bajo /api/. Pruebe con un cliente de pruebas los códigos 200, 201, 400, 404 y 204 y documéntelos en una tabla con capturas.

Respuesta:

CategoriaViewSet y LibroViewSet son ModelViewSet, registrados en un DefaultRouter y montados bajo /api/biblioteca/. Con el cliente de pruebas de Django se obtuvieron los cinco códigos pedidos:

Operación	**Código**
GET /api/biblioteca/libros/ (lista)	200
POST con datos válidos	201
POST con año 1200 (viola la regla)	400
GET /api/biblioteca/libros/99999/	404
DELETE del libro creado	204

biblioteca/api_views.py (líneas 1-36)

```python
from rest_framework import filters, viewsets
from rest_framework.decorators import action
from rest_framework.response import Response

from .models import Categoria, Libro, Prestamo
from .serializers import (CategoriaSerializer, LibroSerializer,
                          PrestamoSerializer)


class CategoriaViewSet(viewsets.ModelViewSet):
    queryset = Categoria.objects.order_by('nombre')
    serializer_class = CategoriaSerializer


class LibroViewSet(viewsets.ModelViewSet):
    queryset = Libro.objects.order_by('titulo')
    serializer_class = LibroSerializer
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['titulo', 'autor', 'isbn']
    ordering_fields = ['titulo', 'anio_publicacion', 'copias']


class PrestamoViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = PrestamoSerializer

    def get_queryset(self):
        prestamos = Prestamo.objects.select_related('socio', 'libro')
        estado = self.request.query_params.get('estado')
        if estado:
            prestamos = prestamos.con_estado(estado)
        return prestamos

    @action(detail=False)
    def con_multa(self, request):
        prestamos = self.get_queryset().con_multa()
        serializer = self.get_serializer(prestamos, many=True)
```

biblioteca/api_urls.py (líneas 1-9)

```python
from rest_framework.routers import DefaultRouter

from . import api_views

router = DefaultRouter()
router.register('categorias', api_views.CategoriaViewSet, basename='categoria')
router.register('libros', api_views.LibroViewSet, basename='libro')
router.register('prestamos', api_views.PrestamoViewSet, basename='prestamo')

```

config/urls.py (líneas 20-26)

```python
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/biblioteca/', include('biblioteca.api_urls')),
    path('api/', include('vet.api_urls')),  # Laboratorio 08 - API REST
    path('', include('vet.urls')),
    path('biblioteca/', include('biblioteca.urls')),
]
```

```
Cliente de pruebas de Django: metodo, URL, codigo
-------------------------------------------------
GET    /api/biblioteca/libros/                200
POST   /api/biblioteca/libros/                201
POST   /api/biblioteca/libros/                400
GET    /api/biblioteca/libros/99999/          404
DELETE /api/biblioteca/libros/7/              204
```

Ejercicio 12 — Recurso de solo lectura con relaciones anidadas

Cree un recurso de solo lectura de su investigación que muestre datos de una entidad relacionada de forma anidada, con get_queryset() optimizado. Mida el número de consultas antes y después con connection.queries y documente ambos valores.

Respuesta:

PrestamoViewSet es de solo lectura (ReadOnlyModelViewSet) y devuelve cada préstamo con el socio y el libro anidados. Su get_queryset() usa select_related('socio', 'libro'). Con 10 préstamos serializados, las consultas pasaron de 21 a 1.

```
biblioteca/serializers.py  (líneas 29-46)
-----------------------------------------
  29          model = Socio
  30          fields = ['id', 'nombre', 'dni']
  31
  32
  33  class LibroResumenSerializer(serializers.ModelSerializer):
  34      class Meta:
  35          model = Libro
  36          fields = ['id', 'titulo', 'autor']
  37
  38
  39  class PrestamoSerializer(serializers.ModelSerializer):
  40      socio = SocioResumenSerializer(read_only=True)
  41      libro = LibroResumenSerializer(read_only=True)
  42
  43      class Meta:
  44          model = Prestamo
  45          fields = ['id', 'socio', 'libro', 'fecha_prestamo',
  46                    'fecha_devolucion_prevista', 'fecha_devolucion_real',
```

```
biblioteca/api_views.py  (líneas 21-36)
---------------------------------------
  21
  22
  23  class PrestamoViewSet(viewsets.ReadOnlyModelViewSet):
  24      serializer_class = PrestamoSerializer
  25
  26      def get_queryset(self):
  27          prestamos = Prestamo.objects.select_related('socio', 'libro')
  28          estado = self.request.query_params.get('estado')
  29          if estado:
  30              prestamos = prestamos.con_estado(estado)
  31          return prestamos
  32
  33      @action(detail=False)
  34      def con_multa(self, request):
  35          prestamos = self.get_queryset().con_multa()
  36          serializer = self.get_serializer(prestamos, many=True)
```

```
medir_api_n1_bib.py
--------------------
Prestamos serializados: 10
Consultas SIN optimizar  (Prestamo.objects.all())      : 21
Consultas CON select_related('socio', 'libro')         : 1
```

```
GET /api/biblioteca/prestamos/1/
--------------------------------
{
  "id": 1,
  "socio": {
    "id": 1,
    "nombre": "Carlos Ramos",
    "dni": "70123456"
  },
  "libro": {
    "id": 1,
    "titulo": "La ciudad y los perros",
    "autor": "M. Vargas Llosa"
  },
  "fecha_prestamo": "2026-10-31",
  "fecha_devolucion_prevista": "2026-11-14",
  "fecha_devolucion_real": null,
  "estado": "Activo",
  "multa": "0.00"
}
```

Ejercicio 13 — Filtros y acción propia en la investigación propia

Agregue SearchFilter y OrderingFilter a un ViewSet, un filtro por parámetro (?campo=) en get_queryset() y una acción con @action(detail=False) que reutilice un método del QuerySet personalizado de la Semana 7. Documente al menos tres URLs con sus resultados.

Respuesta:

LibroViewSet tiene SearchFilter (título, autor, ISBN) y OrderingFilter (título, año, copias). PrestamoViewSet filtra por ?estado= dentro de get_queryset() y tiene la acción con_multa (@action(detail=False)) que reutiliza PrestamoQuerySet.con_multa().

```
biblioteca/api_views.py  (líneas 11-19)
---------------------------------------
  11      queryset = Categoria.objects.order_by('nombre')
  12      serializer_class = CategoriaSerializer
  13
  14
  15  class LibroViewSet(viewsets.ModelViewSet):
  16      queryset = Libro.objects.order_by('titulo')
  17      serializer_class = LibroSerializer
  18      filter_backends = [filters.SearchFilter, filters.OrderingFilter]
  19      search_fields = ['titulo', 'autor', 'isbn']
```

```
biblioteca/api_views.py  (líneas 29-36)
------------------------------------
  29          if estado:
  30              prestamos = prestamos.con_estado(estado)
  31          return prestamos
  32
  33      @action(detail=False)
  34      def con_multa(self, request):
  35          prestamos = self.get_queryset().con_multa()
  36          serializer = self.get_serializer(prestamos, many=True)
```

```
Filtros, busqueda, ordenamiento y accion con_multa
--------------------------------------------------
GET /api/biblioteca/libros/?search=cosmos  ->  1 resultado(s)
    Cosmos (1980)

GET /api/biblioteca/libros/?ordering=-anio_publicacion  ->  5 resultado(s)
    Breve historia del tiempo (1988)
    Cosmos (1980)
    La ciudad y los perros (1963)
    Los ríos profundos (1958)
    Historia de la República (1939)

GET /api/biblioteca/prestamos/?estado=Atrasado  ->  2 resultado(s)
    Breve historia del tiempo - Diego Salas - Atrasado - S/ 7.50
    Los ríos profundos - Elena Vega - Atrasado - S/ 5.00

GET /api/biblioteca/prestamos/con_multa/  ->  4 resultado(s)
    Breve historia del tiempo - Diego Salas - Atrasado - S/ 7.50
    Los ríos profundos - Elena Vega - Atrasado - S/ 5.00
    Historia de la República - Rosa Medina - Extraviado - S/ 30.00
    Cosmos - Diego Salas - Devuelto - S/ 2.50
```

Ejercicio 14 — Publicar en GitHub

Actualice requirements.txt y README.md describiendo qué recursos se expusieron como API, qué serializers y ViewSets se implementaron y qué filtros y acciones se agregaron. Haga commit y push al repositorio. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

Respuesta:

requirements.txt incluye djangorestframework==3.18.3 y el README.md del repositorio describe la API de biblioteca (recursos, rutas, filtros y la acción con_multa). URL del repositorio: https://github.com/RXDG2908/DesAplEmp-S02

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente serializers.py, api_views.py, las respuestas de la API con su código de estado y el número de consultas antes y después de optimizar).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente.

- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Repositorio: https://github.com/RXDG2908/DesAplEmp-S02

Conclusiones:

Un serializer cumple para la API el papel del Form: convierte los objetos a JSON y valida lo que llega.
Una regla propia como validate_stock se aplica sola en is_valid() y responde con 400 cuando no se cumple.
Con un ModelViewSet y un DefaultRouter se obtiene el CRUD completo sin escribir cada vista ni cada ruta.
Los códigos 200, 201, 400, 404 y 204 le dicen al cliente qué pasó sin tener que leer HTML.
Los serializers anidados repiten el problema N+1; con select_related y prefetch_related se bajó de 24 a 3 consultas.
Los filtros y la acción propia reutilizan el CitaQuerySet de la Semana 7: la API aprovecha el ORM, no lo reemplaza.
La API aún está abierta: falta autenticación y permisos antes de usarla fuera del entorno de desarrollo.
Las páginas HTML de semanas anteriores siguen funcionando junto a la API.